FLOCK ENERGY - DATA SCIENCE TAKE-HOME

REFIT data cleaning and washing-machine disaggregation

Raw House 1 cleaning · washing-machine EDA across 12 homes · seq2point-style XGBoost validated leave-one-house-out

Code: https://github.com/ArpitThakur7/flock-energy-refit

Contents

1. Summary and recommendation
2. Overview, setup and design
3. Data, quality and cleaning
4. EDA, model results and failure analysis

Recommendation: using washing-machine disaggregation at a utility

In short: using nothing but a home's 1-minute smart meter reading, the model spots about 8 out of 10 washes in homes it has never seen before, and its daily estimate of washing-machine energy is 28% more accurate than assuming zero. That's good enough for insights about patterns and for grouping customers, such as when and how often a home washes, or which homes are heavy users. It isn't good enough yet to put an exact washing-machine kWh figure on one customer's bill. These results come from 12 UK homes in the REFIT study (2013 to 2015), testing on each home in turn after training on the other 11.

What it's good for

Use	Why it holds up
Spotting whether, how often and when a home does its washing	81% of washes are caught, and the start-time pattern comes through reliably
Grouping or ranking customers, for example heavy washers or people who wash at peak times	In most homes, washing days stand out clearly from non-washing days
Estimates across many homes, for example the total washing load on a feeder between 8 and 11am	Mistakes cancel out across homes: over all 12, the total energy estimate was within 3%

What it's not ready for

- **Quoting one home's exact washing-machine kWh.** For an individual home, the total can be off by 50% to 100%.
- **Making automatic decisions about, or penalising, an individual customer.** Roughly 6 in 10 predicted washes are really another heating appliance, such as a shower, oven or dishwasher.
- **Homes with unusual machines.** In one test home, the washing machine heated in short bursts and the model missed most of its washes.
- **Data at 15 or 30 minutes.** See the last section below.

Section 4 answers

An opportunity: moving washes out of peak times

What I found. Of all washes, 44% start between 7am and noon, and the single busiest time is Saturday at 9am. Two of the 12 homes already use their delay timers to wash at night: House 19 starts 58% of its washes between 10pm and 1am, and House 1 starts 21% between 3am and 6am. Each wash draws about 2 kW for 10 to 25 minutes. That's a big, short block of demand that can easily be moved, since nearly every modern machine has a delay-start button and no new equipment is needed.

What I'd offer. Customers the model picks out as regular morning- or evening-peak washers would get a personalised nudge, something like: "Your washing machine usually runs around 9am. If you set the delay timer to finish by 7am, you could save about £X a year on an off-peak tariff." Where there's a time-of-use tariff or a demand-flexibility event, the message would tie into that.

How I'd check it worked.

- **Set-up:** a randomised trial. Take the customers the model flags, record 4 to 8 weeks of normal behaviour first, then randomly split them into a group that gets the message and a group that doesn't.
- **Main measure:** the share of each customer's detected washes that start at peak times (for example 7 to 11am and 4 to 7pm), compared before and after and between the two groups. The same model is applied to both groups, so its mistakes affect them equally and the difference between them still holds.
- **Other measures:** peak-time energy use, the 9am load on the local network, and bill savings for customers on time-of-use tariffs.
- **Ground truth:** a quick text or in-app question to a sample of customers, such as "Did you use the delay timer this week?". It checks the result and gives labelled data to improve the model.

Overall consumption won't really change, because a wash uses the same energy whenever it runs. The value is in lowering peak demand and cutting customers' bills, and that's how the trial should be judged.

Limitations

Appliances that look alike. Most of a wash's energy is a heating block of around 2 kW, and kettles, showers, ovens, dishwashers and tumble dryers produce very similar blocks. This is the model's biggest source of error, with about 61% of predicted washes being false alarms, and it's why I'd stick to grouped and pattern-based insights for now.

Missing data. Homes lose 6% to 25% of their minutes to outages, sometimes for weeks at a time. I never fill long gaps, so any figure shown to a customer should be worked out over the days that are actually available and say so (for example, "based on 26 of the last 30 days"). The dataset's own "Issues" flag would have removed about a third of the minutes when washing machines are running, so I didn't use it.

Differences between homes. Homes do anywhere from 2 to 11 washes a week, and a wash uses between 0.24 and 0.72 kWh depending on the machine and programme. A machine unlike any in training, like the one in House 19, gets mostly missed. Before launching in a new market, the model should be checked against a small labelled pilot group there.

Using UK results in India. The REFIT homes are mostly detached UK houses with washing machines that heat their own water. Indian homes differ in ways that matter here:

- Many machines have no heater, including semi-automatic top-loaders and many front-loaders run on cold washes. Without the 2 kW heating block, which is the model's strongest clue, a wash looks like a 200 to 500 W motor pattern that's much harder to pick out.
- The big appliances are different: air conditioners, water heaters (geysers), water pumps and inverter battery charging all produce large blocks of their own.
- Voltage varies more and power cuts are more common.
- Metering is often single-phase and lower resolution.

The overall approach would carry over: the cleaning steps, the wash rule and the leave-one-house-out testing. But the model would need retraining and re-testing on labelled Indian homes, and the wash thresholds would need resetting, for example a lower peak threshold for machines without a heater.

What changes with 15- or 30-minute data

At 15 or 30 minutes, each reading averages over most or all of a wash's 10 to 25 minute heating block. The 2 kW block shrinks into a bump of a few hundred watts, about the same as boiling a kettle a few times or a hob on low. The details that make a wash recognisable, its tail, start and finish, get averaged away, and a wash that starts mid-interval gets split across two readings. Telling washes apart from everything else becomes much harder. I'd expect to reliably catch only long, hot washes during otherwise quiet periods, and to get far more confused during busy mornings and evenings. Estimates would have to move to daily or weekly totals with a range attached, along the lines of "probably 3 to 5 washes this week". The model would also have to rely more on habits (time of day, day of week) and household details, which means it would cope worse with unusual homes. You could still tell a customer roughly when they wash, to the nearest half hour, but not count individual washes or say how much energy each one used.

REFIT data cleaning and washing-machine disaggregation

This is my submission for the Flock Energy data science take-home. It works with the REFIT dataset (20 UK homes, 2013 to 2015) and does three things:

1. Cleans the raw House 1 data and resamples it to 1-minute resolution. The trickiest part turned out to be a daylight-saving bug in the raw timestamps. After fixing it, my cleaned series matches the official cleaned data with a correlation of 0.997.
2. Looks at how washing machines are used across 12 homes, using the official cleaned data. I found 4,566 wash cycles and compared when people wash, how long a cycle takes and how much energy it uses.
3. Estimates washing-machine power from the whole-house meter alone. On homes the model has never seen, it picks up about 8 in 10 washes and gets daily washing-machine energy 28% closer than simply guessing zero.

The write-ups are split into three files:

- DATA.md: what the raw data looks like, what was wrong with it, and how I cleaned it
- RESULTS.md: the washing-machine analysis, the model, how I validated it, and where it fails
- Recommendation.md: a one-page summary for a utility, plus my answers to the Section 4 questions

There is also `Flock_Energy_REFIT_submission.pdf`, which is the same four documents in one file.

Dataset

The data is not stored in this repo (the extracted csvs are around 12 GB). It is the **REFIT Electrical Load Measurements** dataset from the University of Strathclyde, released under CC-BY 4.0.

- Dataset page: https://pureportal.strath.ac.uk/en/datasets/refit-electrical-load-measurements/
- Raw data (.7z, ~700 MB): https://pure.strath.ac.uk/ws/portalfiles/portal/62090177/REFIT_RAW_081116.7z
- Cleaned data (.7z, ~500 MB): https://pure.strath.ac.uk/ws/portalfiles/portal/62090184/CLEAN_REFIT_081116.7z
- Cleaned data readme: https://pure.strath.ac.uk/ws/portalfiles/portal/62090183/CLEAN_READ_ME_081116.txt

A note on the links: the `pureportal.strath.ac.uk/files/...` addresses given in the brief returned "403 Forbidden" when I tried them, but the same files download fine from the `pure.strath.ac.uk/ws/portalfiles/...` addresses above. You don't need to fetch anything by hand, though. `src/get_data.py` downloads both archives and pulls out only the files the analysis uses. If you already have the `CLEAN_House*.csv` files, drop them into `data/clean/` and it will use those instead.

Files used:

- Part 1: `RAW_House1_Part1.csv` and `RAW_House1_Part2.csv`
- Parts 2 and 3: `CLEAN_House{1,2,3,4,5,7,15,16,17,18,19,21}.csv`

How to run it

You need Python 3.10 or newer (I used 3.13 on Windows 11) and about 12 GB of free disk space.

```
pip install -r requirements.txt
python run_all.py
```

That one command downloads the data if it isn't there yet and then runs every step. It takes about 5 minutes on my laptop. XGBoost will use a GPU if it finds one and falls back to the CPU otherwise. The random seed is fixed, though GPU training can still differ in the last decimal place.

You can also run the steps one by one from inside `src/`:

Step	Script	What it produces
Get data	`get_data.py`	`data/raw/` and `data/clean/`
Part 1	`part1_clean_raw.py`	the cleaned House 1 series, `outputs/raw_h1_*.csv`, `figures/p1_*.png`
Part 2	`part2_wm_eda.py`	`outputs/wm_*.csv`, `figures/p2_*.png`
Part 3	`part3_model.py`	`outputs/model_*.csv`, `figures/p3_*.png`
Ablation	`REFIT_NO_TIME=1 python part3_model.py`	the same model without time-of-day features, saved as `outputs/*_notime.csv`
PDF	`python tools/build_report_pdf.py --repo-url <url>`	rebuilds the submission PDF (needs Edge or Chrome)

How the code is laid out:

```
src/config.py          paths, house choices and every threshold, all in one place
src/clean_data.py      loads an official cleaned csv and turns it into a 1-minute cache
src/cycles.py          the rule for spotting a washing-machine cycle
src/part1_clean_raw.py, part2_wm_eda.py, part3_model.py, get_data.py
run_all.py             runs everything in order
figures/, outputs/     plots and tables, committed so you can see results without running anything
```

Main decisions and assumptions

Timestamps. I treat the `Unix` column as the clock. The older raw file turned out to store UK local time as if it were UTC, so I convert it (DATA.md explains how I worked that out). All resampling happens in UTC, and I only switch to local time when looking at what hour people wash.

Resampling. Each 1-minute value is the average of the readings inside that minute. Averaging power keeps energy totals correct. If a minute has no readings at all, it stays empty rather than being filled in.

Gaps. Gaps of up to 5 minutes get filled with the last known value. Anything longer is flagged as an outage and left empty, because I don't think it's defensible to invent hours or days of appliance use.

Bad values. Readings that can't be real are set to missing, not to zero: plug sensors above 4 kW, whole-house readings above 15 kW, negative numbers, and any appliance drawing more than the entire house. Setting them to zero would make them look like "appliance off".

Duplicates. Exact duplicate rows are dropped. When several rows share a timestamp but disagree, I keep the most recent non-missing value for each column.

What counts as the washing machine. I use the channel the readme labels as the washing machine. House 4 has two washing machines, so there I add them together.

I didn't use the dataset's own "Issues" flag to drop data. It sounds sensible, but it turns out to fire on 17 to 47% of the minutes when a washing machine is running. Dropping those minutes would throw away a big share of the washes I'm trying to model. Instead I only drop minutes where the washing machine alone reads higher than the whole house.

What counts as a wash. The machine counts as running above 20 W. Pauses of up to 15 minutes are bridged, and a cycle has to last at least 10 minutes, use at least 0.05 kWh and peak above 200 W.

How I validated the model

I used leave-one-house-out testing over all 12 homes. Each home takes a turn as the test home. While it's the test home, it plays no part in training, early stopping or picking any threshold. Inside the training homes, I hold back the last 20% of each home's timeline for early stopping.

The reason for doing it this way: neighbouring 1-minute windows overlap almost completely, so a random split would leak nearly identical data into the test set and make the model look far better than it is. Testing on a home the model has never seen is also the honest version of the real use case, which is a new customer with no appliance-level metering. I compare against two baselines: predicting "off" all the time, and a typical hour-of-week usage profile.

Trade-offs I made

- I used XGBoost on hand-made window features rather than a neural network like Seq2Point. It trains 12 folds in a few minutes, it's easy to inspect, and it already beats both baselines clearly. The cost is that it probably captures the quiet tail end of a wash less well than a CNN would.
- The washing machine is off over 90% of the time, so I keep every minute near a real wash and a random 10% of the rest, giving those a weight of 10. That keeps training to about a million rows per fold without biasing the model.
- After predicting, I keep only predictions that look like a complete wash (same rule as above). This removes a lot of short false alarms and makes the energy estimate much less biased, but it costs some detection on one unusual machine (House 19).
- My headline metrics are daily energy and per-cycle detection, not per-minute error. Per-minute error rewards predicting zero for an appliance that's almost always off, so I report it but don't lean on it.
- I used 12 of the 20 homes. The other eight have a washing-machine channel that is missing, unclear or changes during the study, and I'd rather leave them out than guess.

AI tools used

I used Claude (Anthropic's assistant, through Claude Code) as a coding partner on this project. It helped me get the data downloading when the official links failed, write and debug the scripts, run the checks that turned up the timestamp bug, the odd sensor values and the problem with the "Issues" flag, and draft the documents. I went through the outputs and decisions myself, and every number and plot in the write-ups comes straight from the code in this repo.

What I'd improve with more time

1. Add a second step that looks at each predicted wash as a whole and decides whether it's real. False alarms are the model's biggest weakness right now, with about 6 in 10 predicted washes being something else.
2. Try a Seq2Point neural network on the same folds and compare.
3. Add more homes once I've checked their channel labels by hand, and let the model adapt to a new home from a few confirmed washes.
4. Give a range instead of a single number for monthly washing-machine energy, so an insight is only shown to a customer when the model is confident.
5. Add unit tests for the cycle rule and the timezone conversion. At the moment they're only checked indirectly, through the comparison with the official data and the plots.

Citation

REFIT data © University of Strathclyde, CC-BY 4.0. Murray, D., Stankovic, L. and Stankovic, V. (2017). An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study. *Scientific Data* 4, 160122.

Data: coverage, quality and cleaning

1. The dataset

I used the REFIT Electrical Load Measurements dataset from the University of Strathclyde (Murray et al., 2017, *Scientific Data* 4:160122), which is released under CC-BY 4.0.

It covers 20 homes around Loughborough in the UK, numbered 1 to 21 (there's no House 14), between September 2013 and July 2015. Each home was monitored for somewhere between 15 and 21 months. Every home has one current clamp on the main supply (the "aggregate", meaning the whole house) and nine plug-in monitors on individual appliances. All readings are active power in watts.

Readings come in roughly every 6 to 8 seconds. The sensors aren't synchronised with each other, so any given reading may be up to 8 seconds old. I worked at 1-minute resolution throughout, using the average of the readings in each minute.

Files I used:

Part	Files	From
1 (cleaning raw data)	`RAW_House1_Part1.csv`, `RAW_House1_Part2.csv`	`REFIT_RAW_081116.7z`
2 and 3 (analysis and model)	`CLEAN_House{1,2,3,4,5,7,15,16,17,18,19,21}.csv`	`CLEAN_REFIT_081116.7z`

The download links in the brief returned "403 Forbidden" for me. The same files are available at `pure.strath.ac.uk/ws/portalfiles/portal/<id>/<filename>`, which is what `src/get_data.py` uses.

Some limitations worth knowing about up front:

- The appliance labels describe what was plugged in when the monitors were installed. Householders sometimes moved them later, and the readme lists several known changes.
- A "television site" or "computer site" monitor can cover several devices on one extension lead.
- Plug monitors only see plug-in appliances. Hard-wired loads like an electric shower, oven or immersion heater only ever show up in the whole-house reading, so they're never labelled.
- The official cleaned files fill short gaps with the previous value and replace plug readings above 4 kW with zero. Long outages simply have no rows.
- There are some long stretches of missing data, especially in February 2014. The readme puts these down to internet, hardware and network failures.

---

2. What I found in the raw House 1 data

These numbers come from `src/part1_clean_raw.py`, which saves them to `outputs/raw_h1_inspection.csv`.

Check	Part1 (Oct 2013 to Oct 2014)	Part2 (Jun 2014 to Jul 2015)
Rows	4,377,877	4,140,886
Timestamps that failed to parse	0	0
Times where the clock goes backwards	1	0
Repeated timestamps	1,004,404	14,755
...of which the whole row is a copy	884,307	393
Typical gap between readings	3 s (99% under 15 s)	7 s (99% under 9 s)
Gaps longer than 5 min / 1 hour	380 / 14	72 / 19
Longest gap	998 hours (41.6 days, 24 Jan to 6 Mar 2014)	208 hours
Missing cells	0% (sensors that weren't reporting were written as 0)	12.8% (written as blank)
Whole-house readings over 15 kW	82 (highest 24,000 W)	104 (highest 29,159 W)
Plug readings over 4 kW	3,038	911
Highest plug reading	98,301 W	98,301 W
Negative values	0	0

The things that stood out:

1. **The older file (Part1) is in local time, not UTC.** It recorded UK clock time but stored it as if it were UTC. Three things gave this away:
    - The single place where the clock jumps backwards is `2013-10-27 01:59:59` to `01:00:01`. That's exactly when the clocks went back that autumn, so the hour from 1am to 2am appears twice.
    - The 1am hour is missing on 30 March 2014, when the clocks went forward.
    - When I lined up raw readings against the official cleaned file, Part1 matched with a one-hour shift from April to October and no shift from November to March. Part2 matched with no shift all year, so Part2 was already in UTC.

    If I'd ignored this, every summer reading would be an hour out, and one autumn hour would be out of order.
2. **The two files overlap from 27 June to 1 October 2014,** and they don't agree there: only 20% of the timestamps they share have the same whole-house reading. The official cleaned file follows Part1 for that period.
3. **There are sensor error codes in the data.** Plug readings of 4,096, 8,192, 16,384, 32,767, 65,535 and 98,301 W are integer overflow or bit errors. A plug socket can't deliver much more than 3 kW. Some glitches are lower and harder to spot. For example, the washing machine monitor at one point reads 2,048 W while the whole house is only drawing 190 W.
4. **Part1 has a huge amount of duplication.** About a quarter of its rows repeat an earlier timestamp, and most are exact copies. Part2 has a different quirk: a single reading is split across several rows, each with some sensors filled in and the others blank.
5. **A few whole-house readings are above 15 kW,** 186 in total, up to 29 kW. That isn't strictly impossible on a 100 A supply, but this home normally peaks at around 10 to 12 kW in short bursts (a kettle or shower), so I treated them as bad readings.

---

3. How I cleaned House 1

Every step and its counts are logged in `outputs/raw_h1_steps.csv`.

Step	What I did	How much it changed
1. Read	Used the `Unix` column as the clock. The `Time` column is just a text copy of it.	No failures
2. Fix the timezone	Converted Part1 from UK local time to UTC. For the hour that appears twice in October, I used the order of rows in the file to tell the first pass (summer time) from the second (winter time). Part2 was left alone.	2,795,917 Part1 rows moved by an hour
3. Handle the overlap	Used Part1 up to its last reading (1 Oct 2014, 00:27 UTC) and Part2 only after that. Part1 has more readings in the overlap and matches the official data.	944,950 Part2 rows dropped
4. Remove impossible values	Plug readings over 4 kW, whole-house readings over 15 kW, and negative values set to missing, not zero, so they don't look like "off".	3,895 plug and 171 whole-house readings
5. Sort and de-duplicate	Sorted by time, dropped exact duplicate rows, and merged rows sharing a timestamp by keeping the latest non-missing value in each column. This also stitches Part2's split rows back together.	884,254 dropped, 131,831 merged
6. Resample	Took the average of each UTC minute.	920,091 minutes
6b. Sanity check	An appliance can't use more than the whole house, so any appliance minute above 1.1 times the whole-house reading plus 50 W was set to missing. This catches glitches under 4 kW.	3,821 appliance-minutes (2,498 of them on the electric heater)
7a. Short gaps	Filled gaps of 5 minutes or less with the last value. A sensor skipping a report usually just means nothing changed, and 5 minutes is short next to any appliance cycle.	812 minutes filled
7b. Long gaps	Left anything longer as missing, marked it in an `outage` column and listed it in `outputs/raw_h1_outages.csv`.	408 outages, 121,355 minutes (13.2% of the period)

There were 384 outages between 5 minutes and an hour long, 12 between an hour and a day, and 12 longer than a day. The longest was 41.6 days (24 January to 6 March 2014), then 8.6 days in May 2015 and 7.5 days in June 2015. I didn't fill any of these. There's no honest way to make up days of appliance use, so they stay missing and later steps skip over them.

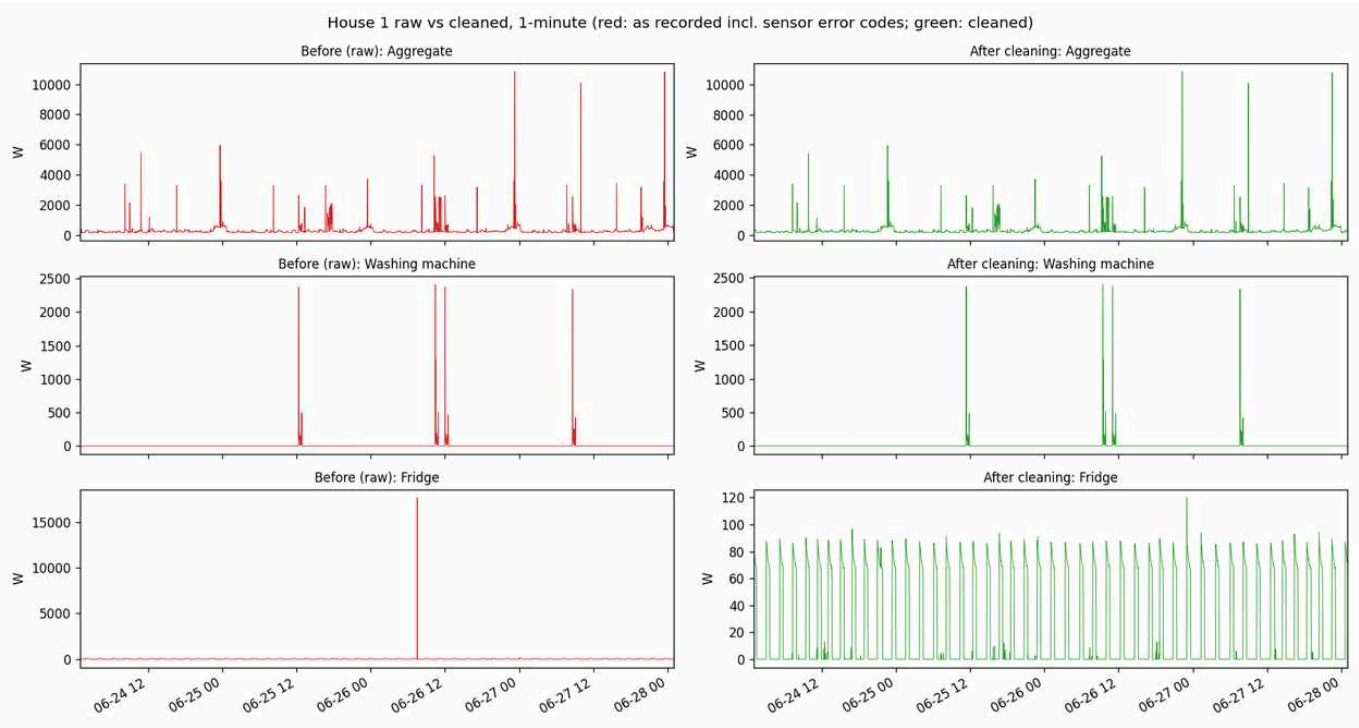
Checking the result. I compared my cleaned whole-house series with the official cleaned version over the 797,815 minutes where both have data:

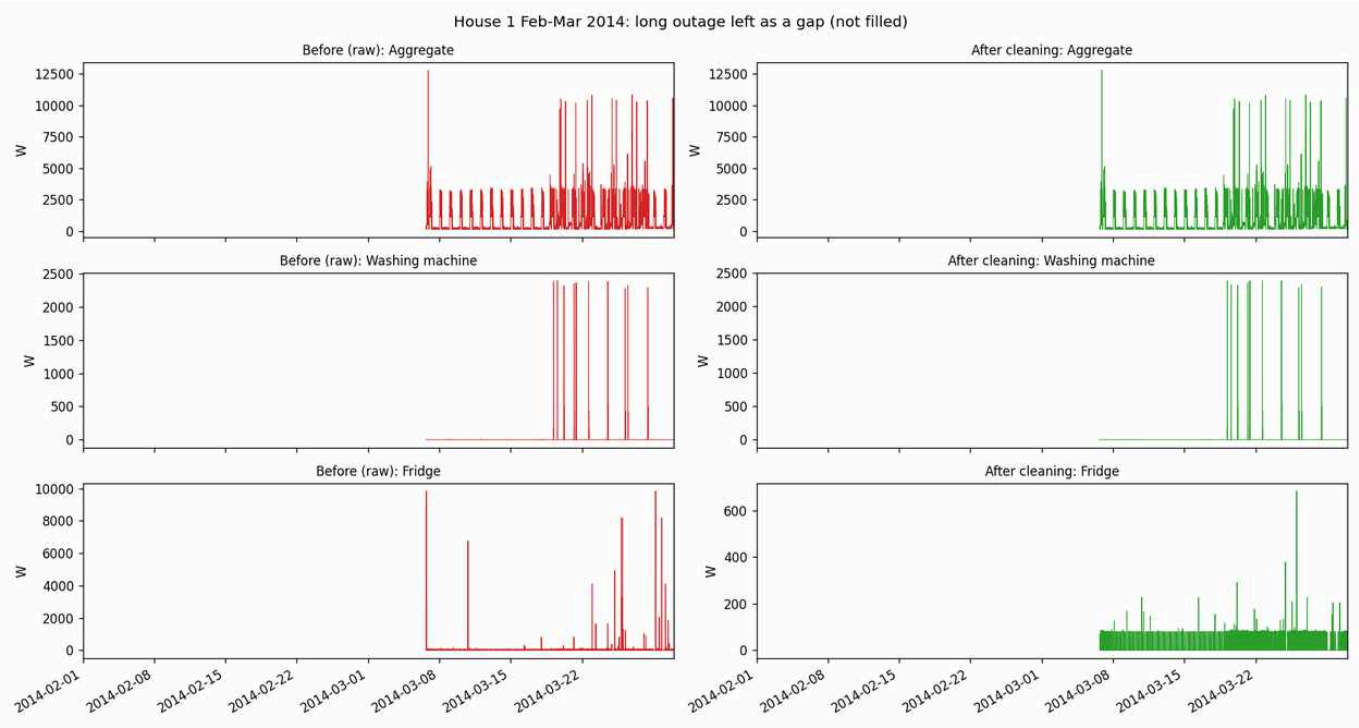
	Correlation with the official data	Average difference
Raw data as recorded	0.768	89.5 W
After my cleaning	0.997	4.4 W

So a fairly simple pipeline gets very close to the official one. Almost all of the improvement comes from fixing the timezone and handling the overlap correctly.

The plots:

- `figures/p1_before_after_week.png` shows four days of the whole-house, washing machine and fridge readings, before and after cleaning. The fake fridge spike of about 17,000 W disappears, the fridge's normal on/off pattern becomes visible, and summer readings move by an hour.
- `figures/p1_before_after_outage.png` covers February and March 2014, with the 41-day outage left as a visible gap.
- `figures/p1_daily_coverage.png` shows, day by day, what share of minutes have a reading.





My cleaning differs from the official version in two ways. I set bad plug readings to missing where the official version sets them to zero. And I didn't try to reassign monitors that householders moved between appliances, since that needs a manual review of each house and the brief says it's out of scope.

---

4. The cleaned data I used for the washing-machine work

Which homes. I used every home where the readme clearly names a washing machine. That came to 12:

House	Washing machine channel	Notes
1	Appliance5	This house also has a washer-dryer (Appliance4). It isn't part of the target, so it will confuse the model.
2, 3, 7, 15, 16, 17, 18, 19, 21	as listed in `src/config.py`	
4	Appliance4 plus Appliance5	Two washing machines, so I add them together. They look the same in the whole-house signal either way.
5	Appliance3	The model number (AEG L99695HWD) shows it's actually a washer-dryer, so its drying runs look different.

I left out Houses 8 and 9 because their washing machine and washer-dryer channels are hard to tell apart. Houses 6, 10, 11, 12, 13 and 20 have a washing-machine channel that's missing, unlabelled or changes partway through. They could be added later after a closer look.

Resampling. Same idea as Part 1: the average of the readings in each UTC minute (see `src/clean_data.py`). Minutes with no readings stay missing, so outages get counted rather than filled. I only use local UK time for the hour-of-day and day-of-week analysis.

Coverage and quality (saved as `outputs/wm_quality.csv`):

House	Period	Missing minutes	Gaps over 1 h	"Issues" flag	Washing machine off	Cycles	Cycles per week
1	Oct 2013 to Jul 2015	12.6%	22	2.1%	98.1%	379	4.8
2	Sep 2013 to May 2015	24.6%	78	2.0%	95.3%	320	4.8
3	Sep 2013 to Jun 2015	11.4%	17	7.7%	94.3%	633	8.1
4	Oct 2013 to Jul 2015	13.7%	212	3.1%	98.5%	256	3.3
5	Sep 2013 to Jul 2015	9.3%	203	8.4%	92.3%	693	8.3
7	Nov 2013 to Jul 2015	12.6%	23	4.6%	91.6%	834	10.9
15	Dec 2013 to Jul 2015	13.7%	198	1.6%	97.1%	249	3.6
16	Jan 2014 to Jul 2015	17.3%	201	1.0%	96.3%	307	4.8
17	Mar 2014 to Jun 2015	6.2%	13	4.3%	97.6%	279	4.4
18	Mar 2014 to May 2015	5.8%	14	5.8%	98.8%	132	2.2
19	Mar 2014 to Jun 2015	6.5%	12	4.1%	96.1%	255	4.1
21	Mar 2014 to Jul 2015	9.4%	15	5.4%	96.9%	229	3.6

"Off" here means 20 W or less. In every home the washing machine is off between 92% and 99% of the time. That imbalance matters a lot when choosing how to measure the model, which I come back to in RESULTS.md.

The dataset's "Issues" flag is biased against washing machines

The readme suggests throwing away rows where the `Issues` column is 1, which marks moments when the nine plug monitors add up to more than the whole-house reading. Before doing that, I checked when it actually fires:

House	1	2	3	4	5	7	15	16	17	18	19	21
Flagged, out of all minutes	2%	2%	8%	3%	8%	5%	2%	1%	4%	6%	4%	5%
Flagged, out of minutes when the washing machine is on	33%	17%	27%	41%	30%	23%	36%	5%	47%	22%	34%	37%
Washing machine alone reads higher than the whole house	5.0%	0.4%	4.5%	2.8%	4.9%	0.9%	1.7%	0.2%	2.6%	0.4%	1.2%	9.0%

The flag goes off five to ten times more often while the washing machine is running. That makes sense once you think about it. Adding up nine unsynchronised monitors during a 2 kW load means a few seconds of lag can easily push the total over the clamp reading. So following the readme's advice would quietly remove a large part of the very thing I'm trying to model.

What I did instead was keep the flagged minutes and only drop minutes where the washing machine on its own reads more than 1.1 times the whole house plus 50 W, which genuinely can't happen.

---

5. Things I'm still not fully comfortable with

- **Moved monitors.** I trust the readme's labels. If someone moved a monitor to a different appliance partway through, the "true" washing machine values are wrong for that stretch.
- **Back-to-back washes.** Because pauses of up to 15 minutes get bridged, two washes run one after the other can merge into one long cycle. You can see this for Houses 5 and 21 in `p2_detected_cycles.png`. It makes some cycles look longer and more energy-hungry than they are, and slightly undercounts washes.
- **Washer-dryers.** House 5's machine, and the extra one in House 1, have drying phases that don't look like a normal wash.
- **Big unlabelled loads.** Showers, ovens and immersion heaters are in the whole-house reading but never metered on their own. They're the main reason the model raises false alarms.
- **Zeros in Part1.** Before Part2, a sensor that wasn't reporting was written as 0, so in Part1 there's no way to tell "off" from "not reporting". This mostly affects the plug monitors, since the whole-house reading is almost never zero.

Results: washing-machine analysis and model

Everything here uses the official cleaned REFIT data at 1-minute resolution, for 12 homes (Houses 1, 2, 3, 4, 5, 7, 15, 16, 17, 18, 19 and 21) over their full recording periods between September 2013 and July 2015. The exact dates for each home are in DATA.md. Between them, these homes ran 4,566 washes that my rule picked up. Every number below comes out of `python run_all.py`.

Part 2: How the washing machines are used

How I decide a wash is happening

In plain terms:

> The machine counts as running in any minute where its average power is above 20 W, comfortably above standby, which is a few watts at most.
>
> A wash has quiet stretches while it soaks, drains or pauses between tumbles. So if the machine stops for 15 minutes or less and then starts again, I treat that as the same wash.
>
> To count as a real wash, a run has to last at least 10 minutes, use at least 0.05 kWh and reach at least 200 W at some point. That filters out little blips like the door lock clicking or the display waking up.

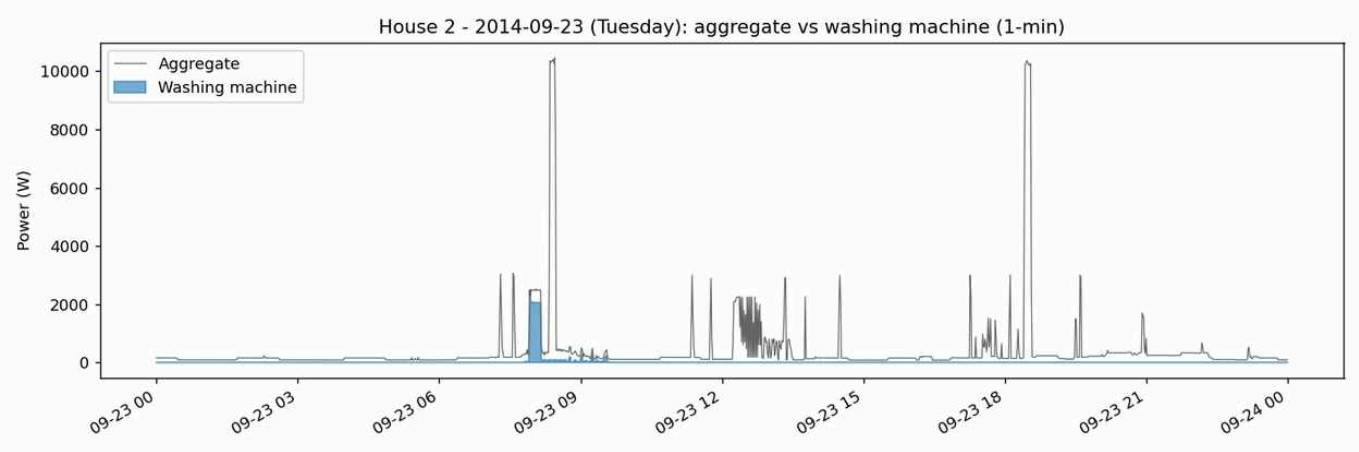
The 20 W cut-off is in line with what other NILM work uses for washing machines (for example Kelly and Knottenbelt, 2015). The code is in `src/cycles.py` and the numbers are in `src/config.py`.

Coverage and data quality

The full table is in DATA.md. In short:

- Each home has 15 to 21 months of data, with 6% to 25% of minutes missing. House 2 is the patchiest, with 78 gaps longer than an hour and one gap of 62 days.
- The washing machine is off (20 W or less) for 92% to 99% of the time. In most homes it reads exactly 0 W when off, though House 5's machine sits at a few watts on standby.
- Homes run anywhere from 2.2 washes a week (House 18) to 10.9 (House 7). The washing machine uses 0.12 to 0.91 kWh a day, which is 1% to 7% of each home's total.

A typical day



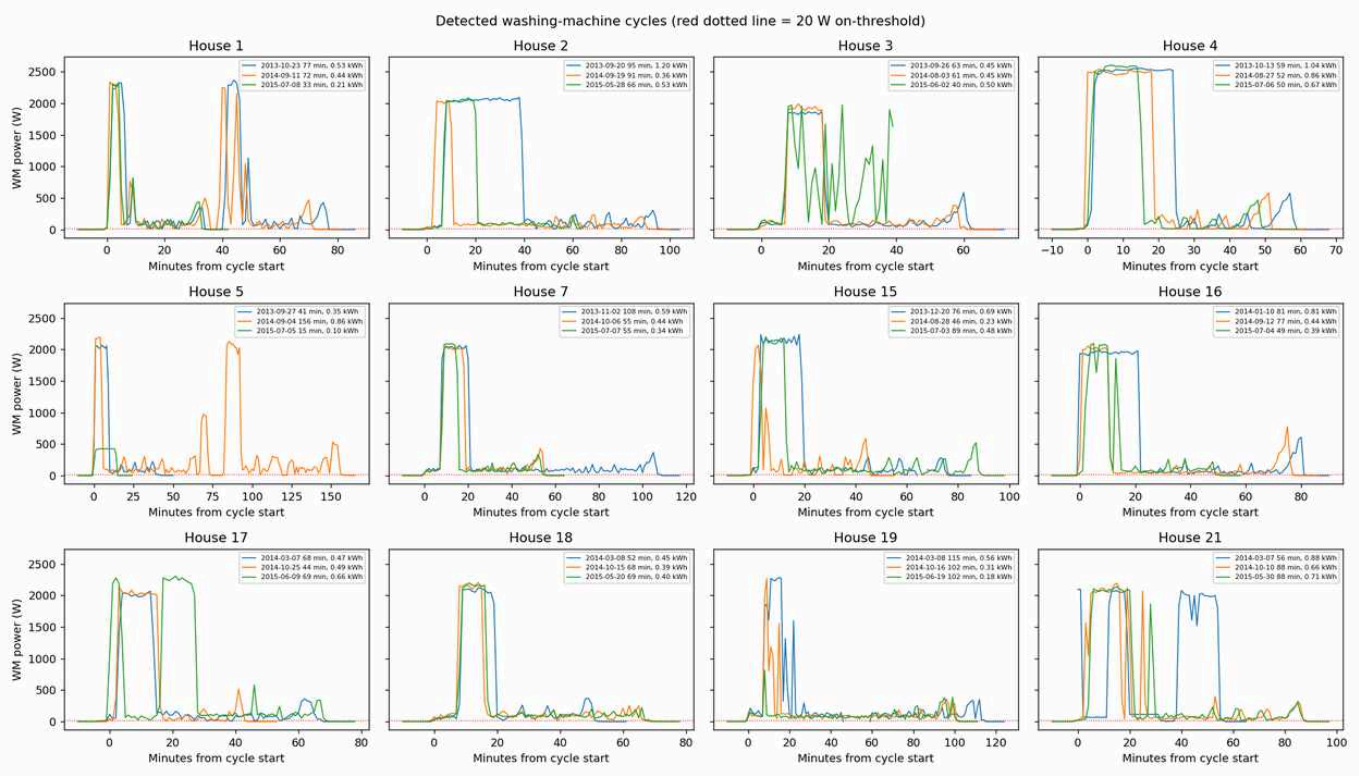
This is House 2 on Tuesday 23 September 2014. I chose it because the data is complete for the whole day and the wash uses a typical amount of energy for that home.

The wash starts just before 8am. It begins with about 10 minutes at around 2 kW, which is the heating element warming the water and where most of the energy goes. After that comes a long, low tail of 50 to 300 W as the drum turns, rinses and

spins. The same day also has two short bursts of about 10 kW (most likely an electric shower) and several 3 kW spikes that look like a kettle.

What this tells me is that a wash's energy is dominated by one block of heating, and that block looks a lot like plenty of other appliances. The long tail is what makes a wash recognisable, but it's small next to everything else going on in the house.

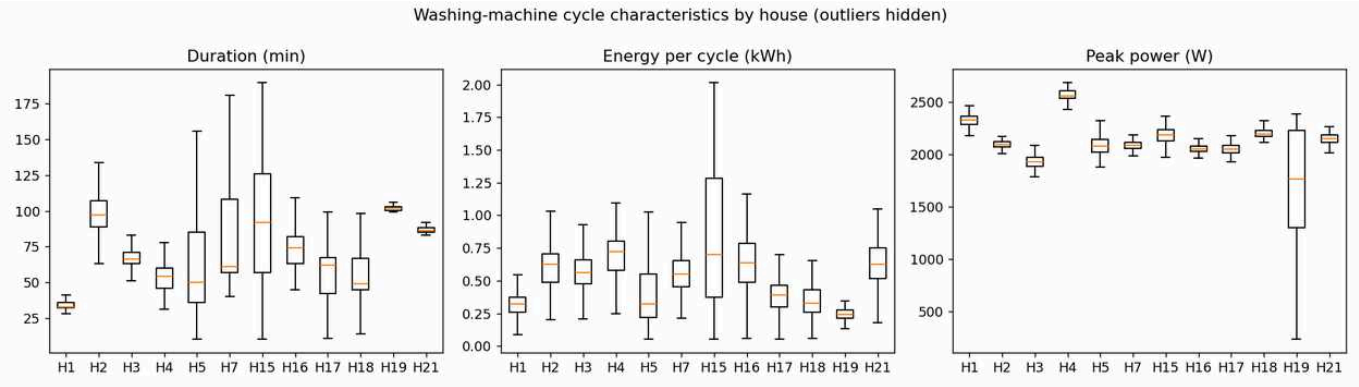
Examples of detected washes



These are three washes from each home (the earliest, one from the middle and the latest), lined up by start time. Nearly all of them share the same shape: a block at 1.8 to 2.6 kW for 5 to 25 minutes, then 30 to 80 minutes of low motor activity.

A few show two heating blocks. Those are either hotter or longer programmes, or two washes run back to back that my 15-minute rule merged (House 5 on 4 September 2014, House 21 on 7 March 2014). House 19 stands out: its machine heats in short bursts instead of one solid block, which turns out to matter a lot for the model.

Comparing homes

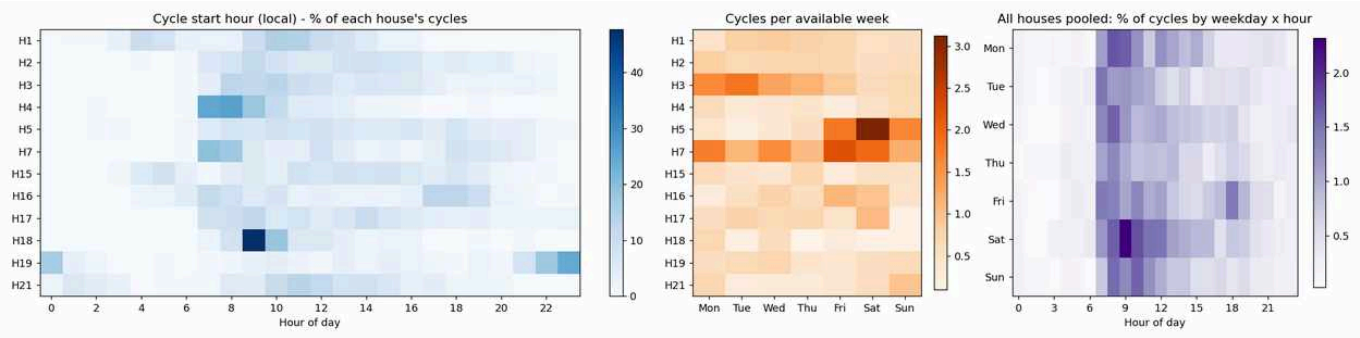


House	Washes	Typical length (min)	Typical energy (kWh)	Typical peak (W)	Share at weekends	Most common start time
1	379	33	0.32	2,327	23%	10am
2	320	97	0.63	2,091	24%	9am
3	633	66	0.56	1,929	16%	10am
4	256	54	0.72	2,556	36%	8am
5	693	50	0.32	2,076	58%	10am
7	834	61	0.55	2,083	29%	7am
15	249	92	0.70	2,181	29%	11am
16	307	74	0.64	2,050	30%	5pm
17	279	62	0.39	2,046	28%	9am
18	132	49	0.33	2,191	15%	9am
19	255	102	0.24	1,759	29%	11pm
21	229	86	0.63	2,145	43%	11am

"Typical" means the median. Peak power barely varies, sitting around 2 kW in almost every home, because that's set by the heating element. Length and energy per wash vary about threefold between homes. House 1's machine runs short, light programmes (about 33 minutes and 0.32 kWh), while House 4 averages 0.72 kWh a wash.

So the programme people pick, mostly how hot they wash, matters about as much as how often they wash. A home doing a few hot washes can easily use more than one doing lots of cool ones.

When people wash



Across all homes, 44% of washes start between 7am and noon, with the busiest time being 8 to 10am, and there's a smaller bump in the early evening between 5 and 7pm. Two homes are very different:

- House 19 starts 58% of its washes between 10pm and 1am.
- House 1 starts 21% of its washes between 3am and 6am.

Both look like people using the delay timer, probably to make the most of a cheaper night-time rate such as Economy 7.

By day of the week, most homes are fairly even. House 5 does most of its washing from Friday to Sunday (58% at weekends), while Houses 3 and 7 wash most at the start of the week. The single busiest slot across everyone is Saturday at 9am.

Part 3: Estimating washing-machine use from the whole-house meter

What the model has to do

Given only the whole-house reading, estimate how much power the washing machine is using each minute. I used all 12 homes over their full periods, which comes to 8.4 million usable minutes after removing minutes with missing data or impossible washing-machine readings.

The model

The core idea comes from sequence-to-point learning (Zhang et al., 2018): to estimate the washing machine at a given minute, look at a window of whole-house readings around it, so the model can see how a wash starts and how it ends.

Zhang and colleagues used a neural network. I used XGBoost on features built from the window instead, because it's faster, easier to explain, and model complexity isn't what's being assessed here.

The features all come from the whole-house reading in a window from 30 minutes before to 30 minutes after:

- the readings themselves at 2-minute steps;
- how far above its usual background level the house is, which takes out the difference between homes with more or less always-on load;
- rolling averages, spreads, minimums and maximums over 5 to 61 minutes;
- how much of the window is spent above 1.5 kW;
- how jumpy the signal is;
- the hour of day and whether it's a weekend. I test whether these help further down.

The washing machine is off most of the time, so for training I keep every minute within an hour of a real wash and a random 10% of all the others. Those other minutes count ten times as much, so the model still learns the true balance between on and off.

After predicting, I clip the estimate so it can't be negative or exceed the whole-house reading, set anything under 20 W to zero, and then keep only predictions that form something that looks like a full wash under the same rule as Part 2. That last step removes lots of short false alarms.

How I tested it

I used leave-one-house-out testing. Each of the 12 homes takes a turn as the test home. While it's the test home, it isn't used for training, for deciding when to stop training, or for choosing any setting. Within the 11 training homes, I hold back the last 20% of each home's timeline to decide when to stop training.

The reason this matters: two neighbouring minutes share 60 out of 61 minutes of their windows. Shuffle the data randomly and you get near-identical examples in both training and test, which makes a model look much better than it really is. Testing on a home the model has never seen avoids that completely. It's also the realistic situation for a utility: a new customer with only a smart meter and no appliance monitors.

What I compared it against

- **Always off:** predict 0 W every minute. Because the machine is off over 90% of the time, this is surprisingly hard to beat on simple per-minute error.
- **Typical weekly pattern:** for each hour of each day of the week, the average washing-machine power across the training homes. It knows roughly when people tend to wash, but nothing about the actual house.

How I measured it, and why

Measure	Why it's useful
Daily energy error (kWh per day)	How far off the estimated washing-machine energy is each day. This is closest to what a bill insight needs.
Total energy error (%)	Whether the model tends to over- or under-estimate over the whole period.
Share of washes caught, and share of predicted washes that are real	What you need to trust anything based on counting or timing washes.
Minute-by-minute F1 (on means over 20 W)	Whether it gets the on/off state right each minute.
Average error per minute, overall and while running	The standard measure in this field. It's dominated by the off time and favours "always off", so I report it but don't rely on it alone.

Results averaged over the 12 unseen homes

Model	Daily energy error (kWh/day)	Total energy error (avg. size)	Minute F1	Washes caught	Predicted washes that are real	Avg. error per minute (W)	Avg. error while running (W)
Always off	0.336	100%	0.00	0%	n/a	**16.3**	463
Typical weekly pattern	0.384	71%	0.10	0%	n/a	31.2	440
XGBoost	0.249	**40%**	0.39	81%	39%	21.9	**313**
XGBoost + wash filter (final)	**0.242**	**40%**	**0.43**	**81%**	**39%**	20.5	318
Final, without time-of-day features	0.244	44%	0.42	80%	38%	21.1	322

And for each home, using the final model:

Test home	Actual kWh	Predicted kWh	Total error	Daily error: model	Daily error: always off	Minute F1	Washes caught	Predicted washes that are real
1	127	120	-5%	0.15	0.20	0.35	81%	42%
2	195	220	+13%	0.25	0.32	0.35	91%	26%
3	322	520	+62%	0.45	0.52	0.49	89%	36%
4	185	80	-57%	0.21	0.29	0.59	98%	59%
5	344	214	-38%	0.42	0.53	0.21	64%	27%
7	470	497	+6%	0.27	0.76	0.62	98%	49%
15	196	82	-58%	0.21	0.34	**0.81**	86%	**88%**
16	206	161	-22%	0.26	0.38	0.36	92%	37%
17	106	94	-11%	0.14	0.23	0.55	91%	45%
18	48	97	+103%	0.15	0.11	0.38	90%	25%
19	65	35	-46%	0.12	0.14	0.19	**17%**	16%
21	106	171	+62%	0.26	0.22	0.24	78%	20%

Adding up all 12 homes, the predicted washing-machine energy comes within 3% of the real total, because the over- and under-estimates largely cancel out.

What I take from this

The model beats both baselines on everything a utility would actually care about. Daily energy error is 28% lower than "always off" and lower in 10 of the 12 homes. It catches 81% of real washes in homes it has never seen before, and its minute-by-minute F1 is four times that of the typical weekly pattern.

The one measure where "always off" wins is average error per minute (16 W against 21 W). That's expected: when the machine is off 92% to 99% of the time, every false alarm costs more than simply predicting zero everywhere. It's a well-known weakness of that measure for appliances that are rarely on, and it's why I don't lead with it.

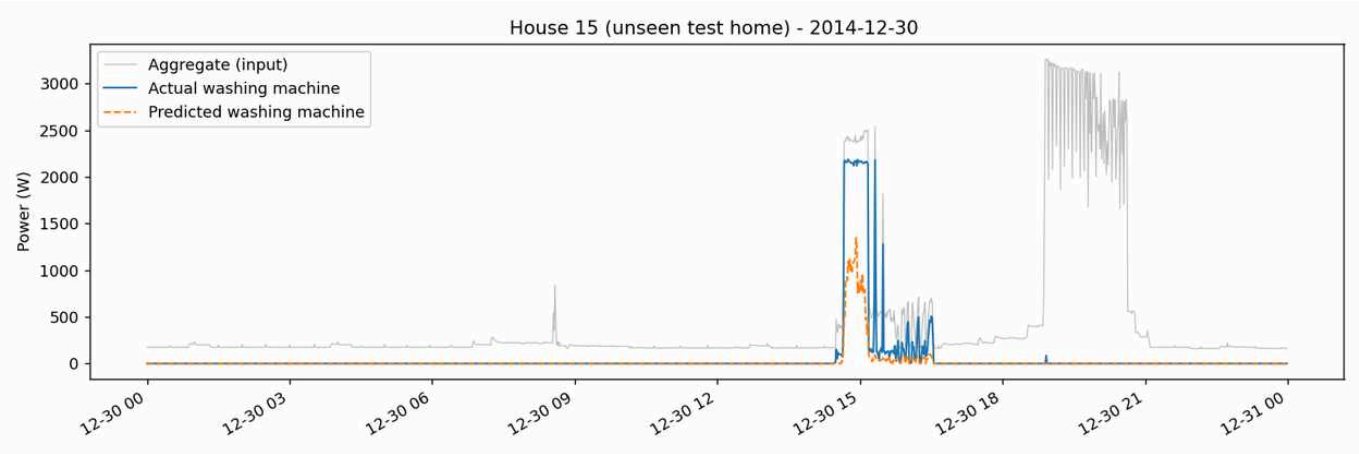
The real weakness is false alarms. Only about 39% of the washes the model reports are real, because other heating appliances produce the same kind of 2 kW block. For a single home, the total energy estimate can be off by 50% to 100% either way. So this model is better suited to grouped or ranked insights than to quoting an exact figure to one customer.

Taking away the hour and weekend features barely changes anything (F1 goes from 0.43 to 0.42). The model is recognising what a wash looks like in the whole-house signal, not memorising when people tend to wash, which is why it holds up in new homes. The features it leans on most (`outputs/model_feature_importance.csv`) are how far above background the house is, the 5-minute minimum (which separates a sustained load from a quick spike), and how long the house spends above 1.5 kW over 15 to 61 minutes.

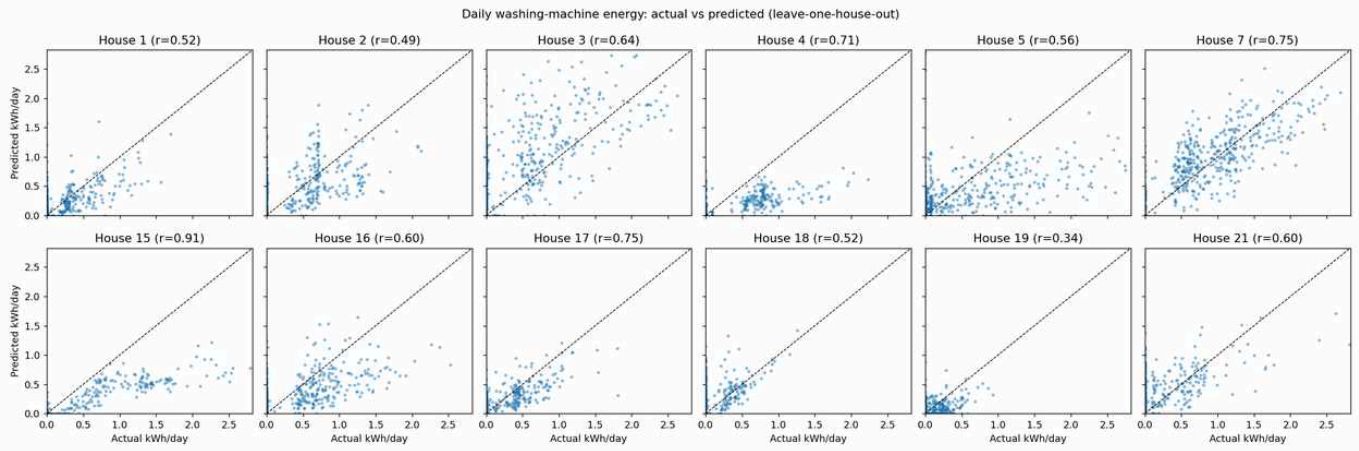
The wash filter mainly removes made-up energy. The average total error moves from 16% too high to almost exactly right (+0.6%), and minute F1 improves from 0.39 to 0.43.

Actual vs predicted on homes the model hasn't seen

A good example is House 15 on 31 January 2015 (`figures/p3_actual_vs_pred_h15.png`). The model gets the heating block and its timing almost exactly right. It partly misses the low tail afterwards, which is why House 15's energy comes out too low overall.

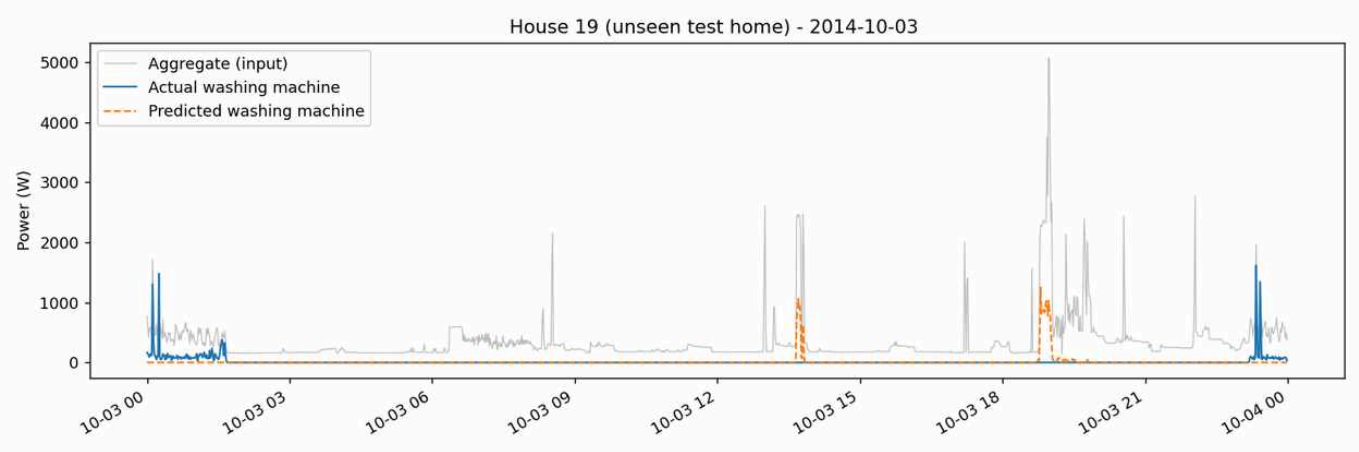


Here is daily energy for every test home (`figures/p3_daily_energy_scatter.png`); a perfect model would put every point on the dashed line. The model separates washing days from non-washing days well in most homes (correlations of 0.5 to 0.86), but it pulls the big washing days down towards the middle.

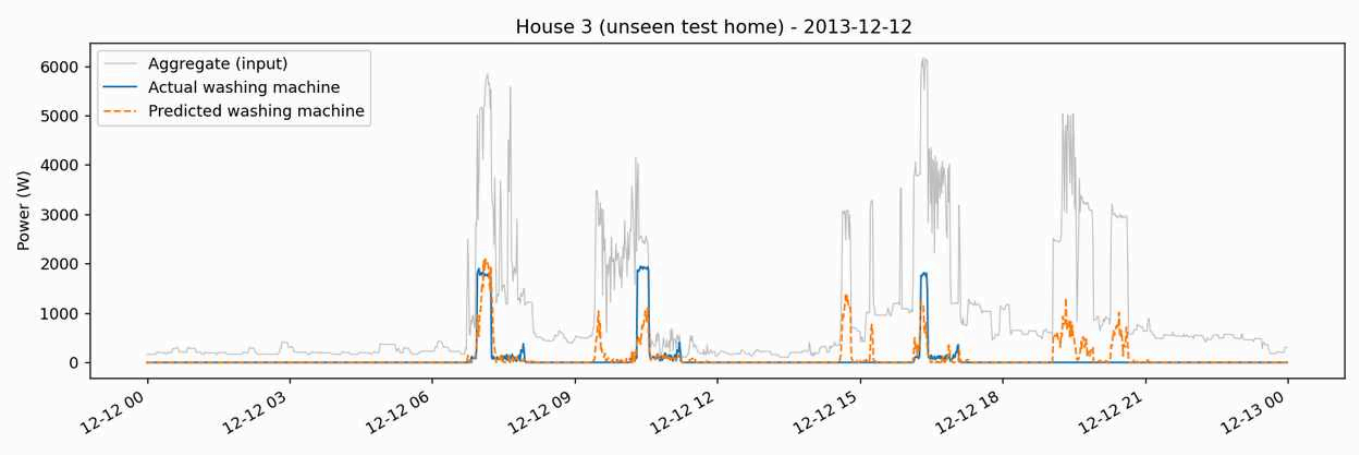


Where it goes wrong

1. A washing machine that doesn't look like the others (House 19). See `figures/p3_actual_vs_pred_h19.png`. This machine heats in short 1.3 to 1.6 kW bursts rather than one long 2 kW block, and mostly runs at night. None of the 11 training homes has a machine like it, so the model misses 83% of its washes. On the same day, it also mistakes two 2.4 kW blocks from something else, probably a dishwasher or oven, for washes (at about 1:40pm and 6:45pm). An unusual machine in a new home is the biggest risk for this kind of model. A few confirmed washes from that home, or simply asking the customer "did you do a wash today?", would help more than anything else.



2. Busy homes with lots of heating appliances (Houses 3 and 21, both 62% too high). See `figures/p3_actual_vs_pred_h3.png`. When the house is full of 2 to 6 kW loads, such as showers, cookers, tumble dryers or a dishwasher heating up, some of them get labelled as washes. Real washes that overlap another big load get underestimated, because the heating block is buried inside something bigger. House 1 has a related problem: its washer-dryer isn't part of the target, but it produces genuinely wash-like cycles, so when the model spots them correctly they still count as false alarms.



Papers I drew on

Paper	What I took from it
Murray, Stankovic and Stankovic (2017), *An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study*, Scientific Data 4:160122	How the dataset was collected, the limits of the plug monitors, and the "Issues" flag, which I then tested and decided not to rely on
Zhang, Zhong, Wang, Goddard and Sutton (2018), *Sequence-to-point learning with neural networks for non-intrusive load monitoring*, AAAI	Estimating the appliance at the middle of a window of whole-house readings, and testing on REFIT homes the model hasn't seen
Kelly and Knottenbelt (2015), *Neural NILM: Deep neural networks applied to energy disaggregation*, ACM BuildSys	A per-appliance "on" threshold (about 20 W for washing machines), training on windows weighted towards real activations, and reporting both energy and on/off measures
Makonin and Popowich (2015), *Nonintrusive load monitoring (NILM) performance evaluation*, Energy Efficiency 8	Why accuracy and per-minute error are misleading for rarely used appliances, and why energy-based and on/off-based measures should be reported together
D'Incecco, Squartini and Zhong (2019), *Transfer learning for non-intrusive load monitoring*, IEEE Transactions on Smart Grid	That testing on other houses is the realistic test, which is why I chose leave-one-house-out

What I'd try next

1. A second step that looks at each predicted wash as a whole (how long it lasts, how much energy it uses, the shape of its tail, time since the last wash) and decides whether it's real. This goes straight at the false alarms, and I'd test it the same way.
2. A Seq2Point neural network on the same test setup, to see whether it picks up the quiet tail end of a wash better than my hand-made features.
3. Bringing in the remaining REFIT homes (6, 8, 9, 10, 13 and 20) after checking their labels by hand, so the model sees more kinds of machine.
4. Giving a range rather than a single number, for example for monthly washing-machine energy, so a utility can choose to show an insight only when the model is confident.